Justin Gould

IS 6503: Principles of Database Management

Spring 2026

Week 16 Assignment

Professor Nayem Rahman, PhD

May 12, 2026

Final Project

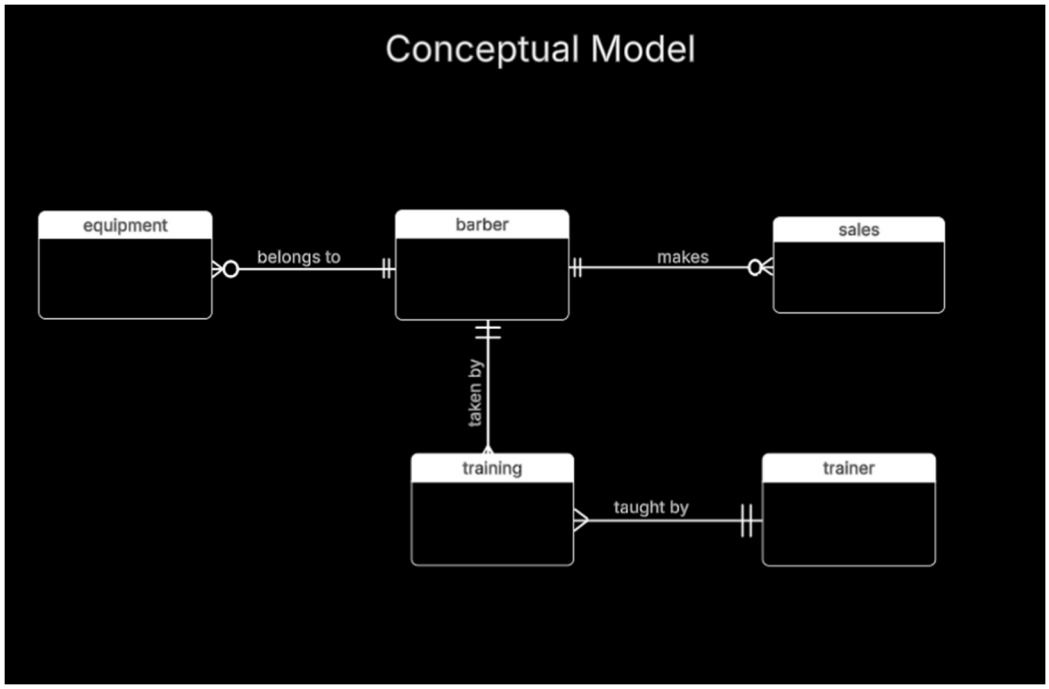
Phase 1: The Business Problem

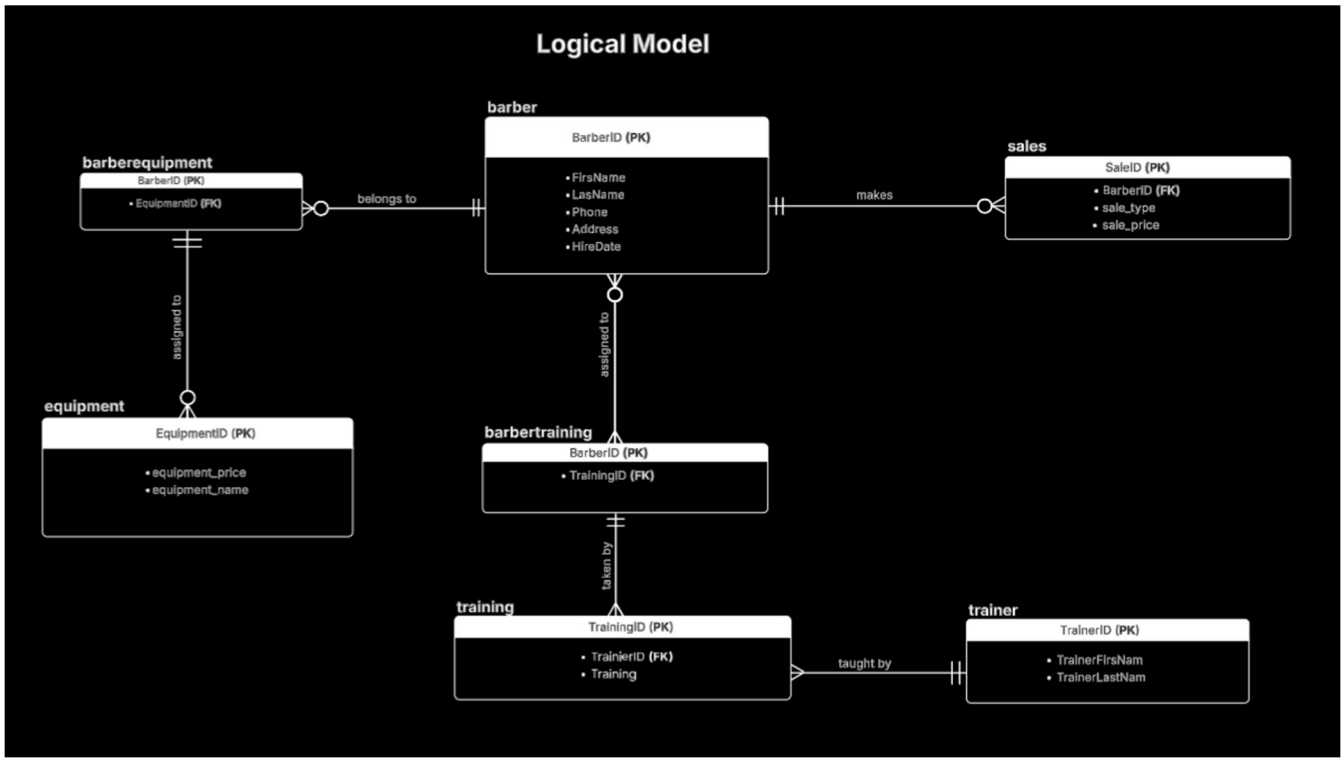
A small barbershop is looking to hire my business – Cutting Edge Analytics. Cutting Edge Analytics will bring structure to the barbershop's data and create a database management system. The shop will provide data that will need to be structured into entities and individual tables. The shop has also requested a data model in the conceptual, logical, and physical form. This data will consist of a variety of sales, employee, equipment, and barber training data. The tools that will be necessary for this company's data engineering tasks will be SQL, Lucidchart, Excel, and Microsoft word.

Phase 2: Conceptual and Logical Data Model

Develop a Conceptual Model. Consider 5 or 6 entities. Make sure you have at least one many-to-many relationship. Explain with data why it's a many-to-many relationship.

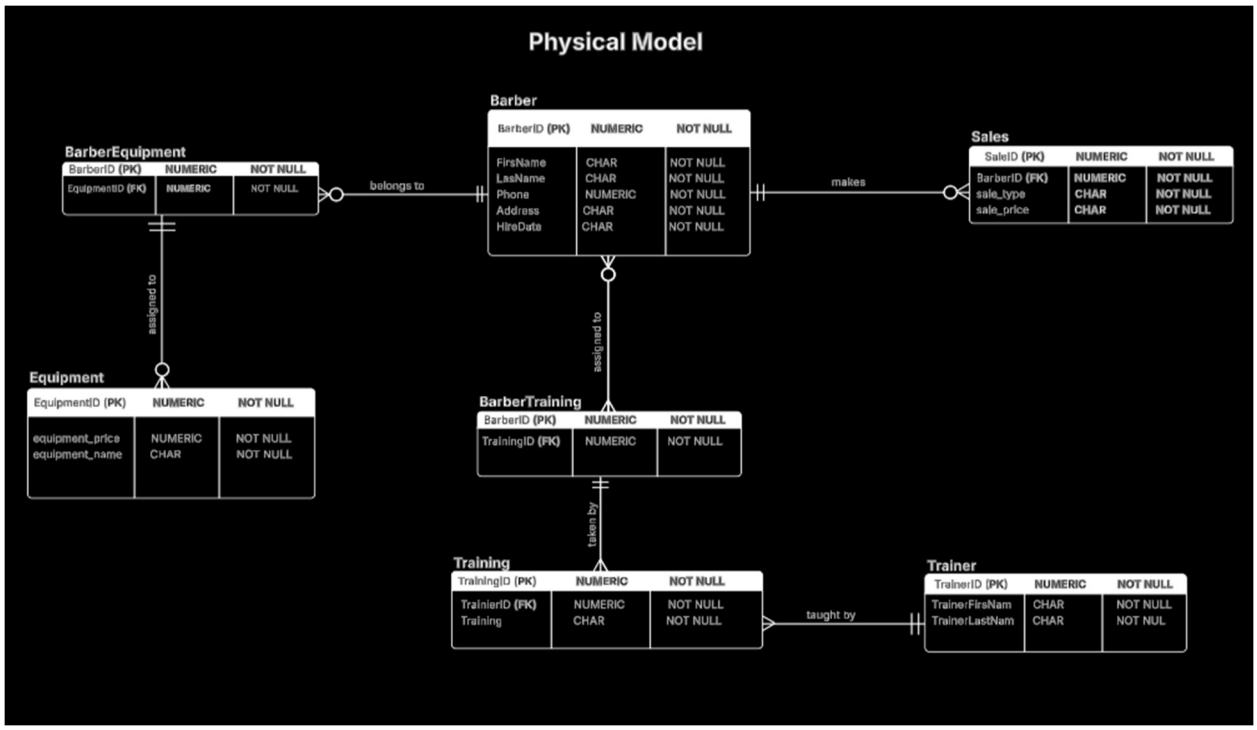
There is a many-to-many relationship between equipment and barber because one piece of equipment can belong to many barbers and one barber can possess many pieces of equipment. There is also a many-to-many relationship between barbers and sales because one barer can offer many sales and many sales can be offered by many different barbers. Lastly, there is a many-to-many relationship between the barber and training table because one barber can attend many trainings and one training can be attended by many different barbers.


Conceptual Model
equipment
belongs to
barber
makes
sales
taken by
training
taught by
trainer


Logical Model
barber
BarberID (PK)
• FirsName
• LasName
• Phone
• Address
• HireDate
barberequipment
BarberID (PK)
• EquipmentID (FK)
belongs to
makes
sales
SaleID (PK)
• BarberID (FK)
• sale_type
• sale_price
assigned to
assigned to
equipment
EquipmentID (PK)
• equipment_price
• equipment_name
barbertraining
BarberID (PK)
• TrainingID (FK)
taken by
training
TrainingID (PK)
• TrainierID (FK)
• Training
taught by
trainer
TrainerID (PK)
• TrainerFIrsNam
• TrainerLastNam

Final Phase: Physical Data Model + SQL Ingestion

5. Develop the physical model based on the Logical Model



6. Create tables using a database system. Insert data into the database tables. You must provide the DDL (CREATE TABLE statements), INSERT statements, and SELECT statements. Details: Create the tables that you have come up with (the table must be based on the Physical Model).
(a) Columns, Primary Key (PK), Data Type, and length, and NULL/NOT NULL need to be implemented, per the Physical Model.

See the code below and ERD above.

(b) Show the table definition (DDL) that you implemented (not in a graphical view).

```sql
-------------------------------------
-- 1. CREATING THE TABLES
-------------------------------------
-- Table: Barber
CREATE TABLE Barber (
    BarberID NUMERIC NOT NULL,
    FirstName CHAR(50) NOT NULL,
    LastName CHAR(50) NOT NULL,
    Phone NUMERIC NOT NULL,
    Address CHAR(100) NOT NULL,
    HireDate CHAR(20) NOT NULL,
    PRIMARY KEY (BarberID)
);

-- Table: Equipment
CREATE TABLE Equipment (
    EquipmentID NUMERIC NOT NULL,
    equipment_price NUMERIC NOT NULL,
    equipment_name CHAR(50) NOT NULL,
    PRIMARY KEY (EquipmentID)
);

-- Table: Trainer
CREATE TABLE Trainer (
    TrainerID NUMERIC NOT NULL,
    TrainerFirstName CHAR(50) NOT NULL,
    TrainerLastName CHAR(50) NOT NULL,
    PRIMARY KEY (TrainerID)
);

-- Table: Training
CREATE TABLE Training (
    TrainingID NUMERIC NOT NULL,
    TrainerID NUMERIC NOT NULL,
    Training CHAR(100) NOT NULL,
    PRIMARY KEY (TrainingID, TrainerID),
    FOREIGN KEY (TrainerID) REFERENCES Trainer(TrainerID)
);

-- Table: Sales
CREATE TABLE Sales (
    SaleID NUMERIC NOT NULL,
    BarberID NUMERIC NOT NULL,
    sale_type CHAR(50) NOT NULL,
    sale_price NUMERIC NOT NULL,
    PRIMARY KEY (SaleID),
    FOREIGN KEY (BarberID) REFERENCES Barber(BarberID)
);

-- Table: BarberEquipment (Bridge Table)
CREATE TABLE BarberEquipment (
    BarberID NUMERIC NOT NULL,
    EquipmentID NUMERIC NOT NULL,
    PRIMARY KEY (BarberID, EquipmentID),
    FOREIGN KEY (BarberID) REFERENCES Barber(BarberID),
    FOREIGN KEY (EquipmentID) REFERENCES Equipment(EquipmentID)
);

-- Table: BarberTraining (Bridge Table)
CREATE TABLE BarberTraining (
    BarberID NUMERIC NOT NULL,
    TrainingID NUMERIC NOT NULL,
    PRIMARY KEY (BarberID, TrainingID),
    FOREIGN KEY (BarberID) REFERENCES Barber(BarberID),
    FOREIGN KEY (TrainingID) REFERENCES Training(TrainingID)
);
```

(c) Insert the complete set of data that you have come up with and show the insert statements used.

```
-- ========================================
-- 2. INSERTING THE DATA
-- ========================================

-- Populate Barbers
INSERT INTO Barber (BarberID, FirstName, LastName, Phone, Address, HireDate)
VALUES (1, 'Siera', 'Andrews', 2109990899, '800 West Street', 'January 15, 2001'),
       (2, 'Philip', 'Martin', 2109081243, '809 Oak Haven', 'March 18, 2017'),
       (3, 'Daniel', 'Niles', 2109879008, '8908 Main St', 'November 14, 2003'),
       (4, 'Miles', 'Garrett', 2100088979, '15 Wilderness Drive', 'November 15, 2005'),
       (5, 'Jerod', 'Brown', 2100983012, '29 Short Ave', 'March 23, 2016');

-- Populate Equipment
INSERT INTO Equipment (EquipmentID, equipment_name, equipment_price)
VALUES (1, 'Chairs', 500),
       (2, 'Clippers', 200),
       (3, 'Computers', 300),
       (4, 'Internet Router', 600),
       (5, 'Shampoo Products', 80),
       (6, 'Barber Capes', 25);

-- Populate Trainers
INSERT INTO Trainer (TrainerID, TrainerFirstName, TrainerLastName) VALUES
(1, 'Amy', 'Sullivan'),
(2, 'Jonathan', 'Alonzo'),
(3, 'Alfred', 'Potts'),
(4, 'Derek', 'Williams'),
(5, 'Wesley', 'Matthews'),
(6, 'Sarah', 'Burnes'),
(7, 'Cade', 'Johnson'),
(8, 'Candra', 'Lopez');

-- Populate Sales
INSERT INTO Sales (SaleID, BarberID, sale_type, sale_price)
VALUES (1, 1, 'Regular Haircut', 45),
       (2, 2, 'Shave', 15),
       (3, 3, 'Low Fade', 40),
       (4, 4, 'Mid Fade', 45),
       (5, 5, 'High Fade', 45),
       (6, 1, 'Trim', 25),
       (7, 2, 'Scissor Cut', 15),
       (8, 3, 'Buzz Cut', 20);
```

```sql
-- Populate Training
INSERT INTO Training (TrainingID, TrainerID, Training)
VALUES (1, 1, 'Welcome Training'),
       (2, 2, 'Ethics'),
       (3, 3, 'Conflict Management'),
       (3, 7, 'Conflict Management'),
       (4, 4, 'How to Fade Hair'),
       (5, 5, 'Customer Care'),
       (6, 6, 'Practicum'),
       (6, 8, 'Practicum');

-- Populate Barber-Equipment
INSERT INTO BarberEquipment (BarberID, EquipmentID)
VALUES (1, 1), (1, 2), (1, 5), (1, 6),
       (4, 1), (4, 2), (4, 5), (4, 6),
       (2, 1), (2, 2), (2, 5), (2, 6),
       (5, 1), (5, 2), (5, 5), (5, 6);

-- Populate Barber-Training
INSERT INTO BarberTraining (BarberID, TrainingID)
VALUES (1, 1), (1, 3),
       (4, 1), (4, 5), (4, 6),
       (2, 4), (2, 2),
       (5, 1), (5, 2), (5, 4);
```

7. Create a variety of SQL queries to retrieve data from one or many tables:

1. Retrieve the data from each table by using the SELECT * statement and order by PK column(s). Show the output. Make sure you show the print screen of the complete set of rows and columns. The rows must be ordered by PK column(s).

```sql
-- Barber Table
SELECT *
FROM barbershop.Barber
ORDER BY BarberID;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

BarberID	FirstName	LastName	Phone	Address	HireDate
1	Siera	Andrews	2109990899	800 West Street	January 15, 2001
2	Philip	Martin	2109081243	809 Oak Haven	March 18, 2017
3	Daniel	Niles	2109879008	8908 Main St	November 14, 2003
4	Miles	Garrett	2100088979	15 Wilderness Drive	November 15, 2005
5	Jerod	Brown	2100983012	29 Short Ave	March 23, 2016
NULL	NULL	NULL	NULL	NULL	NULL

```sql
-- Equipment Table
SELECT *
FROM barbershop.Equipment
ORDER BY EquipmentID;
```

Result Grid | Filter Rows: | Edit:

EquipmentID	equipment_price	equipment_name
1	500	Chairs
2	200	Clippers
3	300	Computers
4	600	Internet Router
5	80	Shampoo Products
6	25	Barber Capes
NULL	NULL	NULL

```sql
-- Trainer Table
SELECT *
FROM barbershop.Trainer
ORDER BY TrainerID;
```

Result Grid | Filter Rows: | Edit:

TrainerID	TrainerFirstName	TrainerLastName
1	Amy	Sullivan
2	Jonathan	Alonzo
3	Alfred	Potts
4	Derek	Williams
5	Wesley	Matthews
6	Sarah	Burnes
7	Cade	Johnson
8	Candra	Lopez
NULL	NULL	NULL

```sql
-- Training Table
SELECT *
FROM barbershop.Training
ORDER BY TrainingID, TrainerID;
```

Result Grid | Filter Rows: | Edit:

TrainingID	TrainerID	Training
1	1	Welcome Training
2	2	Ethics
3	3	Conflict Management
3	7	Conflict Management
4	4	How to Fade Hair
5	5	Customer Care
6	6	Practicum
6	8	Practicum
NULL	NULL	NULL

```sql
-- Sales Table
SELECT *
FROM barbershop.Sales
ORDER BY SaleID;
```

Result Grid | Filter Rows: | Edit:

SaleID	BarberID	sale_type	sale_price
1	1	Regular Haircut	45
2	2	Shave	15
3	3	Low Fade	40
4	4	Mid Fade	45
5	5	High Fade	45
6	1	Trim	25
7	2	Scissor Cut	15
8	3	Buzz Cut	20
NULL	NULL	NULL	NULL

```sql
-- BarberEquipment Table
SELECT *
FROM barbershop.BarberEquipment
ORDER BY BarberID, EquipmentID;
```

Result Grid | Filter Rows: | Edit:

BarberID	EquipmentID
1	1
1	2
1	5
1	6
2	1
2	2
2	5
2	6
4	1
4	2
4	5
4	6
5	1
5	2
5	5
5	6
NULL	NULL

```sql
-- BarberTrainng Table
SELECT *
FROM barbershop.BarberTraining
ORDER BY BarberID, TrainingID;
```

Result Grid | Filter Rows: | Edi

BarberID	TrainingID
1	1
1	3
2	2
2	4
4	1
4	5
4	6
5	1
5	2
5	4
NULL	NULL

2. Write an SQL involving the junction table and two other related tables. You must use the INNER JOIN to connect with all three tables. The database that you created must be included in your SQL queries.

```
175     -- ========================================
176     -- 4. USING THE JUNCTION TABLE (INNER JOIN)
177     -- ========================================
178 •   SELECT
179         b.BarberID,
180         b.FirstName,
181         b.LastName,
182         be.EquipmentID,
183         e.equipment_name
184     FROM barbershop.barber b
185     INNER JOIN barbershop.barberequipment be
186         ON b.BarberID = be.BarberID
187     INNER JOIN barbershop.equipment e
188         ON be.EquipmentID = e.EquipmentID
189     ORDER BY b.barberID;
190
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BarberID	FirstName	LastName	EquipmentID	equipment_name
1	Siera	Andrews	1	Chairs
1	Siera	Andrews	2	Clippers
1	Siera	Andrews	5	Shampoo Products
1	Siera	Andrews	6	Barber Capes
2	Philip	Martin	1	Chairs
2	Philip	Martin	2	Clippers
2	Philip	Martin	5	Shampoo Products
2	Philip	Martin	6	Barber Capes
4	Miles	Garrett	1	Chairs
4	Miles	Garrett	2	Clippers
4	Miles	Garrett	5	Shampoo Products
4	Miles	Garrett	6	Barber Capes
5	Jerod	Brown	1	Chairs
5	Jerod	Brown	2	Clippers
5	Jerod	Brown	5	Shampoo Products
5	Jerod	Brown	6	Barber Capes

Result 40

3. Write an SQL by including two or more tables and using the LEFT OUTER JOIN. Show the results and sort the results by key field(s). Interpret the results compared to what an INNER JOIN does.

```
191     -- ================================================
192     -- 5. USING THE JUNCTION TABLE (LEFT OUTTER JOIN)
193     -- ================================================
194 •   SELECT
195         b.BarberID,
196         b.FirstName,
197         b.LastName,
198         be.EquipmentID,
199         e.equipment_name
200     FROM barbershop.barber b
201     LEFT OUTER JOIN barbershop.barberequipment be
202         ON b.BarberID = be.BarberID
203     LEFT OUTER JOIN barbershop.equipment e
204         ON be.EquipmentID = e.EquipmentID
205     ORDER BY b.barberID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BarberID	FirstName	LastName	EquipmentID	equipment_name
1	Siera	Andrews	1	Chairs
1	Siera	Andrews	2	Clippers
1	Siera	Andrews	5	Shampoo Products
1	Siera	Andrews	6	Barber Capes
2	Philip	Martin	1	Chairs
2	Philip	Martin	2	Clippers
2	Philip	Martin	5	Shampoo Products
2	Philip	Martin	6	Barber Capes
3	Daniel	Niles	NULL	NULL
4	Miles	Garrett	1	Chairs
4	Miles	Garrett	2	Clippers
4	Miles	Garrett	5	Shampoo Products
4	Miles	Garrett	6	Barber Capes
5	Jerod	Brown	1	Chairs
5	Jerod	Brown	2	Clippers
5	Jerod	Brown	5	Shampoo Products
5	Jerod	Brown	6	Barber Capes

The difference between the LEFT and OUTER JOIN is that the Left join keeps all the records from the Left table which in this case was Barber and the Inner join keeps only those that match. You can see the difference in the results where the LEFT JOIN kept Daniel Niles–the only employee without any equipment and the INNER JOIN did not.

---

4. Write a single-row subquery. Show the results and sort the results by key field(s). Interpret the output.

```
208     -- ================================================
209     -- 6. SINGLE-LINE SUB QUERY
210     -- ================================================
211
212 •   SELECT BarberID
213     FROM barbershop.barberequipment
214     WHERE EquipmentID IN (SELECT EquipmentID FROM barbershop.equipment WHERE equipment_price >= 250)
215     ORDER BY BarberID;
216
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BarberID
1
2
4
5

This query returns the BarberID's of all the barbers that are using equipment that costs less than $250. It is basically showing you who is using the 'cheap equipment.'

---

5. Write a multiple-row subquery. Show the results and sort the results by key field(s). Interpret the output.

```
218     -- ================================================
219     -- 6. MULTI-LINE SUB QUERY
220     -- ================================================
221     -- Trainings with ONLY one trainer
222 •   SELECT *
223     FROM barbershop.Trainer
224     WHERE TrainerID IN (SELECT TrainingID
225                         FROM barbershop.Training
226                         WHERE Training = ('Welcome Training')
227                            OR (Training LIKE '%How%')
228                         GROUP BY trainingID HAVING count(TrainingID) <=1)
229     ORDER BY TrainerID ;
230
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

TrainerID	TrainerFirstName	TrainerLastName
1	Amy	Sullivan
4	Derek	Williams
NULL	NULL	NULL

This query returns the names of a trainers that are teaching the Welcome Training course OR any such training that contains the word 'How' in its title.

---

6. Write an SQL to aggregate the results by using multiple columns in the SELECT

clause. Interpret the output.

```
231     -- ============================================
232     -- 7. A aggregated SQL query w/ multiple columns
233     -- ============================================
234 •   SELECT
235         b.FirstName,
236         b.LastName,
237         t.training,
238         count(b.barberID) as Trainings_attended
239     FROM barbershop.barber b
240     LEFT OUTER JOIN barbershop.barbertraining bt
241         ON b.BarberID = bt.BarberID
242     LEFT OUTER JOIN barbershop.training t
243         ON bt.TrainingID = t.TrainingID
244     WHERE t.training IS NOT NULL
245     GROUP BY b.firstname, b.lastname, t.training
246     ORDER BY count(b.barberID);
247
```

Result Grid | Filter Rows: | Export: | Wrap Cell

FirstName	LastName	training	Trainings_attended
Siera	Andrews	Welcome Training	1
Philip	Martin	Ethics	1
Philip	Martin	How to Fade Hair	1
Miles	Garrett	Welcome Training	1
Miles	Garrett	Customer Care	1
Jerod	Brown	Welcome Training	1
Jerod	Brown	Ethics	1
Jerod	Brown	How to Fade Hair	1
Siera	Andrews	Conflict Management	2
Miles	Garrett	Practicum	2

This query shows all the training and how many times they were attended. I used the WHERE clause to filter out one employee who was not involved in any training. The COUNT() function is used to count which trainings were attended X number of times.

---

7. Write a subquery using the NOT IN operator. Show the results and sort the results by key field(s). Interpret the output.

```
246     -- ================================================
247     -- 8. SUB QUERY WITH NOT IN
248     -- ================================================
249 •   SELECT BarberID
250     FROM barbershop.barberequipment
251     WHERE EquipmentID NOT IN (SELECT EquipmentID
252                               FROM barbershop.equipment
253                               WHERE equipment_price >= 250)
254     ORDER BY BarberID;
255
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BarberID
1
1
1
2
2
2
4
4
4
5
5
5

This query returns the BarberID's of all the barbers that are using equipment that costs more than $250. It basically shows you who is using the 'expensive equipment.'

8. Write a query using a CASE statement. Show the results and sort the results by key field(s). Interpret the output.

```
268     -- ================================================
269     -- 9. USE A CASE STATEMENT
270     -- ================================================
271 •   SELECT
272         equipment_name,
273         equipment_price,
274         CASE WHEN equipment_price < 100 THEN "Low Cost Equipment"
275             WHEN equipment_price >= 100 AND equipment_price <= 300 THEN "Medium Cost Equipment"
276             WHEN equipment_price > 300 THEN "High Cost Equipment" END AS Cost_Buckets
277     FROM barbershop.equipment
278     order by equipment_price
279
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

equipment_name	equipment_price	Cost_Buckets
Barber Capes	25	Low Cost Equipment
Shampoo Products	80	Low Cost Equipment
Clippers	200	Medium Cost Equipment
Computers	300	Medium Cost Equipment
Chairs	500	High Cost Equipment
Internet Router	600	High Cost Equipment

This query is effectively bucketing the results of the equipment by their cost. For equipment that is less than $100 it is categorized as Low Cost, equipment that is between $100 and $300 is categorized as Medium Cost, and anything over 300 is categorized as High Cost.

9. Write a query using the NOT EXISTS operator. Show the results and sort the results by key field(s). Interpret the output.

```
280     -- ================================================
281     -- 10. USE NOT EXISTS
282     -- ================================================
283 •   SELECT *
284     FROM barbershop.barber
285     WHERE NOT EXISTS (SELECT 1
286                       FROM barbershop.barbertraining
287                       WHERE barbertraining.BarberID = barber.BarberID)
288     ORDER BY BarberID;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

BarberID	FirstName	LastName	Phone	Address	HireDate
3	Daniel	Niles	2109879008	8908 Main St	November 14, 2003
NULL	NULL	NULL	NULL	NULL	NULL

This query effectively shows you all the employees that did not attend any training classes. Since Daniel Niles ID never appears in the BarberTraining table it is shown here in the results.

---

10. Write a subquery using the NOT NULL operator in the inner query. Show the results and sort the results by key field(s). Interpret the output.'

```
291     -- ================================================
292     -- 11. USE NOT NULL INSIDE THE SUB QUERY
293     -- ================================================
294 •   SELECT *
295     FROM barbershop.barber
296     WHERE barberID IN (SELECT barberID
297                        FROM barbershop.barberequipment
298                        WHERE barber.BarberID IS NOT NULL)
299     ORDER BY BarberID;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cel

BarberID	FirstName	LastName	Phone	Address	HireDate
1	Siera	Andrews	2109990899	800 West Street	January 15, 2001
2	Philip	Martin	2109081243	809 Oak Haven	March 18, 2017
4	Miles	Garrett	2100088979	15 Wilderness Drive	November 15, 2005
5	Jerod	Brown	2100983012	29 Short Ave	March 23, 2016
NULL	NULL	NULL	NULL	NULL	NULL

This query shows you all the barbers that used equipment. The subquery is filtering out those that did not have any equipment needs.

SUBMISSION REQUIREMENT:
Submit the final report with details of each phase of the project, starting from the problem statement. That means you must submit all the above 7 steps (including the requirements of Update 1, Update 2, and steps 5, 6, and 7).

Provide a summary (two paragraphs) of your work at the very end of the report.

Upon completion of this project, my organization, Cutting Edge Analytics (CEA), was able to bring structure to the small barbershop's data needs. CEA did this in three phases, phase 1 consisted of identifying the business need which was to bring structure to the barbershops data. Phase 2 consisted of modeling their databases entities in the form of a Conceptual and Logical data model. The Conceptual model had 5 entities, and the Logical Model grew to 7 entities after 2 junction entities were added. Lastly, the data was modeled in the physical form and ingested into SQL.

After the data was fully modeled it was ingested into MySQL using standard DDL language. Cutting Edge Analytics ran several queries both advanced and easy-to-read for the owners of the barbershop to verify the data's integrity. The data passed all the integrity tests with all the Primary Keys and Foreign Keys being in their respective tables. Thanks to the junction tables, good data modeling, and sound SQL logic, the small barbershops business problem was fully resolved.

Upload your submission to the Week 16 Final Project Report Dropbox on Canvas.